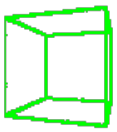
Grenoble INP - Ensimag, UGA

Introduction to Visual Computing

TP2

Image Formation



GOMEZ ROSALES Laura Sofía
LUGO NOMESQUE José Francisco

Prof. Sergi Pujades
Prof. Julien Pansiot

October 5, 2025

Contents

1 Rigid Transformation **2**

2 Orthogonal projection and Perspective Projection **3**
2.1 Pinhole and uv projection 3

3 Using Depth **4**

4 Output examples **5**

5 Bonus **11**

Projecting images 3D into 2D plane is a process that requires different mathematical operations such as rotation, translation and dimensional projection, the pipeline consist of the following steps:

- Rigid Transformation.
- Orthogonal projection.
- Perspective Projection.

1 Rigid Transformation

First step of rendering pipeline for 3d image into 2d image, the point cloud should be center at the origin, then a 4x4 matrix is computed using the rotation and translation parameters.

Then, a Rigid Transformation was performed to the 3D object to place it in the desired position and orientation, by applying algebraic operations with the matrix to every point of the object.

```
    for (int i = 0; i < N_v; i++) {
    // Convert homogeneous coordinates
        px = points[i].x;
        py = points[i].y;
        pz = points[i].z;
        pw = 1.0f;

        // Eq 1: Apply the transformation (Rigid transformation)
        x_prime = TransMatrix[0]*px + TransMatrix[1]*py
                  + TransMatrix[2]*pz + TransMatrix[3]*pw;

        y_prime = TransMatrix[4]*px + TransMatrix[5]*py
                  + TransMatrix[6]*pz + TransMatrix[7]*pw;

        z_prime = TransMatrix[8]*px + TransMatrix[9]*py
                  + TransMatrix[10]*pz + TransMatrix[11]*pw;

        t_w = TransMatrix[12]*px + TransMatrix[13]*py
              + TransMatrix[14]*pz + TransMatrix[15]*pw;
```

2 Orthogonal projection and Perspective Projection

2.1 Pinhole and uv projection

Moving the image from 3-dimensions to 2-dimensions there are different projection types. The main difference between Orthagonal and Perspective projections, is that the first one use parallel lines in the 3D world remain parallel in the 2D image, due to this in the second one the **pinhole transformation** and **uv projection** were used with the 3D point's X and Y coordinates which were divided by their depth (z').

```
        // Project the point
         if (orthogonal) {
             // Camera plane coordinates
             x_cam = x_prime;
             y_cam = y_prime;

             // Pixel coordinates of the projected point (UV projection)
             u = (x_cam / alpha_u) + u_0;
             v = (y_cam / alpha_v) + v_0;

         } else {
             // Eq 3: Camera plane coordinates
             // (pinhole projection)
             x_cam = x_prime / (1 + z_prime / f);
             y_cam = y_prime / (1 + z_prime /f);

             // Eq 4: Pixel coordinates of the projected point
             // (UV projection)
             u = (x_cam / alpha_u) + u_0;
             v = (y_cam / alpha_v) + v_0;
         }
```

3 Using Depth

The Z Buffer is used to apply the depth of the 3d points regarding the camera's position and the parameters given to the program.
Initializing Depth buffer outside the for cycle:

```
    if (use_depth) {
        printf("Using Depth Buffer\n");

        z_buffer = (float *) malloc(width * height * sizeof(float));

        for (int i = 0; i < width * height; i++){
            z_buffer[i] = FLT_MAX;
        }
    }
```

Inside the for cycle: 'Is this new point closer to the camera than the point currently stored in the Z-buffer for pixel (u, v)?'

Then colors or RGB channels of each point that fulfill the condition are stored in the pixmap struct.

```
        if (use_depth){
            depth = z_prime;
            pixel_index = v_int * width + u_int;

            if (depth < z_buffer[pixel_index]){
                z_buffer[pixel_index] = depth;
                image.pixmap[pixel_index].red = points[i].r;
                image.pixmap[pixel_index].green = points[i].g;
                image.pixmap[pixel_index].blue = points[i].b;
            }
```

4 Output examples

Using the pipeline applied to two models (Cube and Human) and setting parameters such as extrinsic and intrinsic camera values to rotate and translate the 3D object, below there are comparisons between the baseline and the result projections of changing each parameter.

- Baseline image



(a) Cube (b) Human

Figure 1: 1 300 200 150 100 0.01 0.01 0 0 0 0 0 -2 0

Generated images take extrinsic parameters (gamma, beta, alpha, `T_x`,`T_y` and `T_z`) to rotate and translate the model respectively using the camera in the same point to verify the different operations that could be applied.

Then with intrinsic values were modified (f, `u_0`, `v_0`, `alpha_u`,`alpha_v`) to verify what is the result of edit the distance to the model, the principal coordinates and the scale of the model.

- Shift the principal point horizontally `u_0`



(a) Cube (b) Human

Figure 2: 1 300 200 **200** 100 0.01 0.01 0 0 0 0 0 -2 1

- Making the horizontal pixel size smaller to stretch the image horizontally `alpha_u`



(a) Cube (b) Human

Figure 3: 1 300 200 150 100 **0.005** 0.01 0 0 0 0 0 -2 1

- Moving the objet farther `T_z`



(a) Cube (b) Human

Figure 4: 1 300 200 150 100 0.01 0.01 0 0 0 0 0 **-5** 1

- Translating the object sideways diagonal `T_x`



(a) Cube (b) Human

Figure 5: 1 300 200 150 100 0.01 0.01 0 0 0 **0.5** 0 -2 1

- Rotating the object around Y axis `beta`



(a) Cube (b) Human

Figure 6: 1 300 200 150 100 0.01 0.01 0 **0.5** 0 0 0 -2 1

- Pinhole image with a rotated view so the perspective is obvious



(a) Cube (b) Human

Figure 7: **1** 300 200 150 100 0.01 0.01 **0.4 0.4** 0 0 0 -2 1

- Orthogonal image with `f=0`



(a) Cube (b) Human

Figure 8: **0** 300 200 150 100 0.01 0.01 0.4 0.4 0 0 0 -2 1

The pinhole projection shows perspective, where parallel lines converge. The orthogonal projection preserves parallel lines and object size regardless of depth

- Z-buffer is necessary for correct rendering of solid objects



(a) 1 300 200 150 100 0.01 0.01 0 0 0 0 0 -2 **1**



(b) 1 300 200 150 100 0.01 0.01 0 0 0 0 0 -2 **0**

Figure 9: Cube



(a) 1 300 200 150 100 0.01 0.01 0 0 0 0 0 -2 **1**



(b) 1 300 200 150 100 0.01 0.01 0 0 0 0 0 -2 **0**

Figure 10: Human

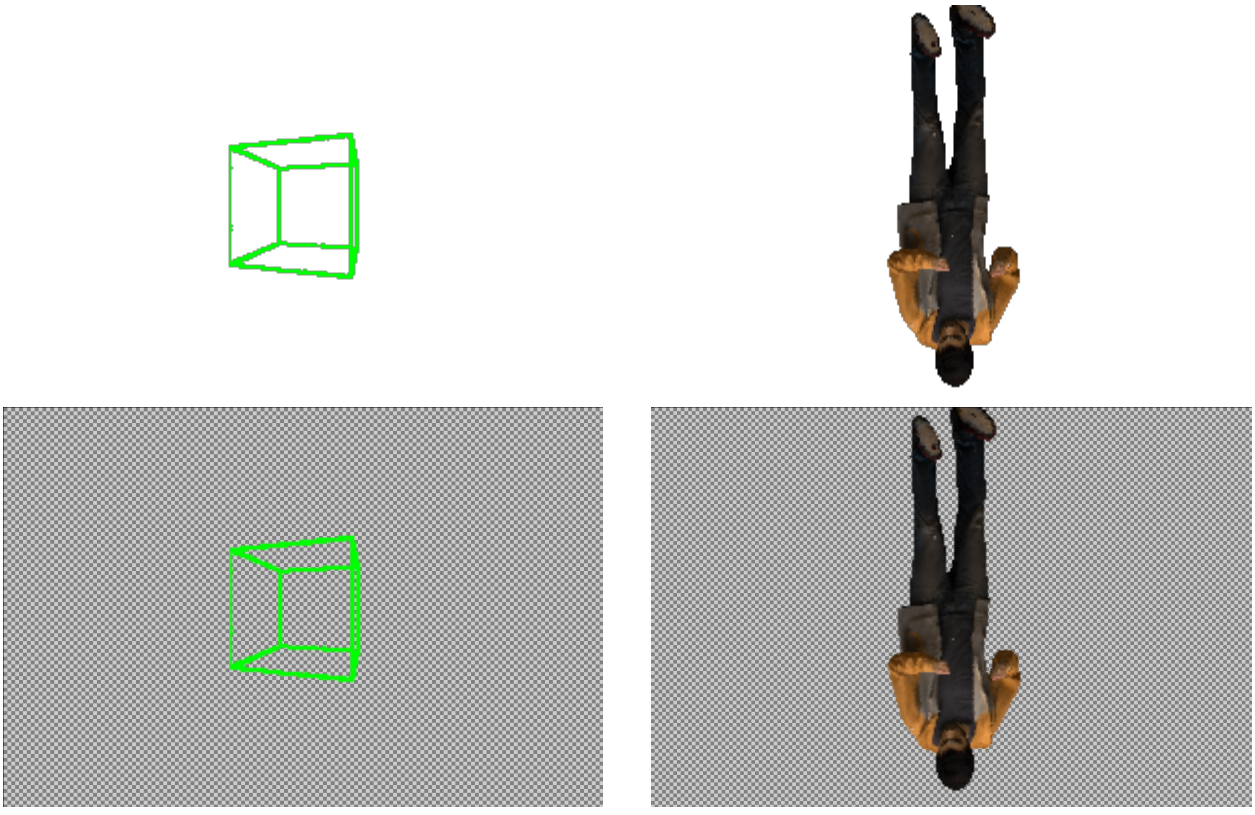
5 Bonus

The projections could be written into PPM images, although it is possible write PAM images taking into account the transparency, in order to obtain the projection with alpha values.

The `project_points_bonus` file was developed in a separate document, then each time that original code takes RGB values, now there is an alpha value that is added to the `pixel_rgba` struct, then the function to write the PAM document were used variables such as filename where is store, width, height and the rgba buffer that was filled before.
Compiling the file is done in the same steps that the original one, but execution should be done with the new compiled file.

```
./project_points_bonus models/frameCube.off
  1 300 200 150 100 0.01 0.01 0 0.3 0 0 0 -2 1

% ffmpeg -i results/image.pam results/image.png
```



(a) Cube (b) Human

Figure 11: 1 300 200 150 100 0.01 0.01 0 0.3 0 0 0 -2 1